

Отчет по курсовому проекту / практической работе №8 по курсу «Алгоритмы и структуры данных»

Студент группы М8О-116БВ-25 Сиухин Ярослав Алексеевич, № по списку 22

Контакты www, e-mail: zaq2007q@gmail.com

Работа выполнена: «03» июня 2026 г.

Преподаватель: Речинская Ангелина Юрьевна

Отчет сдан «__» _____ 2026 г., итоговая оценка _____

Подпись преподавателя _____

Тема	Линейные списки, итераторы и базовые операции над динамической структурой данных.
Цель	Реализовать двусвязный список с барьерным элементом и итератором, а также выполнить операции вставки, удаления, печати, получения длины и дополнения списка.
Задание	Создать список целых чисел, реализовать вставку и удаление по индексу, обход через итератор и дополнение списка копиями заданного значения до длины k.
Файлы	/Users/siuxin_yaroslav/Desktop/projectsLabsC/second_semestr/kp8

Оборудование и программное обеспечение

- Компьютер: Apple Silicon, архитектура arm64.
- Операционная система: macOS 15.7.3.
- Интерпретатор команд: bash 3.2.57.
- Система программирования: Apple clang 17.0.0, стандарт C11.
- Редактор текстов: nano / среда разработки и терминал macOS.

Идея, метод и алгоритм

В основе программы находится двусвязный список с барьерным узлом. Итератор хранит текущий узел и ссылку на список, поэтому операции обхода и изменения списка выполняются через единый интерфейс.

1. Создать структуру списка и барьерный узел.
2. Для вставки по индексу переместить итератор к нужной позиции и вставить новый узел перед текущим.
3. Для удаления по индексу переместить итератор к удаляемому узлу и перенастроить связи соседних элементов.
4. Для печати пройти список от первого узла до конца через итератор.
5. Для дополнения списка сравнить текущую длину с k и добавить в конец недостающее число копий указанного значения.
6. Перед завершением освободить все выделенные узлы и служебные структуры.

Сценарий выполненной работы

Файл: lab3.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

typedef struct {
    Node *barrier;
    int size;
} List;

typedef struct {
    Node *current;
    List *list;
} Iterator;

List* create_list() {
    List *list = (List*)malloc(sizeof(List));
    if (!list) return NULL;
    Node *bar = (Node*)malloc(sizeof(Node));
    if (!bar) {
        free(list);
        return NULL;
    }
    bar->prev = NULL;
    bar->next = NULL;
    bar->data = 0;
    list->barrier = bar;
    list->size = 0;
    return list;
}

void destroy_list(List *list) {
    if (!list) return;
    Node *cur = list->barrier->next;
    while (cur) {
        Node *tmp = cur;
        cur = cur->next;
        free(tmp);
    }
    free(list->barrier);
    free(list);
}

int list_size(List *list) {
    return list->size;
}

Iterator* create_iterator(List *list) {
    Iterator *it = (Iterator*)malloc(sizeof(Iterator));
    if (!it) return NULL;
    it->list = list;
    it->current = NULL;
    return it;
}

void destroy_iterator(Iterator *it) {
    free(it);
}

int iterator_valid(Iterator *it) {
    return (it && it->current != NULL && it->current != it->list->barrier);
}

void iterator_first(Iterator *it) {
    if (!it || !it->list) return;
    it->current = it->list->barrier->next;
}
```

```

void iterator_next(Iterator *it) {
    if (iterator_valid(it))
        it->current = it->current->next;
}

int iterator_get(Iterator *it) {
    if (iterator_valid(it))
        return it->current->data;
    return 0;
}

void iterator_insert_before(Iterator *it, int value) {
    if (!it || !it->list) return;
    List *list = it->list;
    Node *new_node = (Node*)malloc(sizeof(Node));
    if (!new_node) return;
    new_node->data = value;

    if (it->current == NULL) {
        Node *last = list->barrier;
        while (last->next)
            last = last->next;
        new_node->next = NULL;
        new_node->prev = last;
        last->next = new_node;
    } else {
        Node *prev_node = it->current->prev;
        new_node->next = it->current;
        new_node->prev = prev_node;
        prev_node->next = new_node;
        it->current->prev = new_node;
    }
    list->size++;
}

void iterator_delete(Iterator *it) {
    if (!iterator_valid(it)) return;
    List *list = it->list;
    Node *to_delete = it->current;
    Node *prev_node = to_delete->prev;
    Node *next_node = to_delete->next;

    if (prev_node)
        prev_node->next = next_node;
    if (next_node)
        next_node->prev = prev_node;

    free(to_delete);
    list->size--;
}

void print_list(List *list) {
    Iterator *it = create_iterator(list);
    printf("Список (длина %d): ", list_size(list));
    if (list_size(list) == 0) {
        printf("нуст\n");
        destroy_iterator(it);
        return;
    }
    for (iterator_first(it); iterator_valid(it); iterator_next(it)) {
        printf("%d ", iterator_get(it));
    }
    printf("\n");
    destroy_iterator(it);
}

void insert_at_index(List *list, int index, int value) {
    if (!list || index < 0 || index > list->size) {
        fprintf(stderr, "Ошибка: неверный индекс для вставки\n");
        return;
    }
    Iterator *it = create_iterator(list);
    if (index == list->size) {
        it->current = NULL;
    } else {
        iterator_first(it);
        for (int i = 0; i < index; i++) {

```

```

        iterator_next(it);
    }
}
iterator_insert_before(it, value);
destroy_iterator(it);
}

void delete_at_index(List *list, int index) {
    if (!list || index < 0 || index >= list->size) {
        fprintf(stderr, "Ошибка: неверный индекс для удаления\n");
        return;
    }
    Iterator *it = create_iterator(list);
    iterator_first(it);
    for (int i = 0; i < index; i++) {
        iterator_next(it);
    }
    iterator_delete(it);
    destroy_iterator(it);
}

void extend_list(List *list, int value, int k) {
    if (!list) return;
    int current = list_size(list);
    if (current >= k) {
        printf("Длина списка уже >= %d, дополнение не требуется.\n", k);
        return;
    }
    int to_add = k - current;
    printf("Добавляем %d элементов со значением %d...\n", to_add, value);
    Iterator *it = create_iterator(list);
    it->current = NULL;
    for (int i = 0; i < to_add; i++) {
        iterator_insert_before(it, value);
    }
    destroy_iterator(it);
    printf("Готово.\n");
}

int main() {
    List *list = create_list();
    if (!list) {
        fprintf(stderr, "Ошибка создания списка\n");
        return 1;
    }

    int choice, value, index, k;

    do {
        printf("\n--- Меню ---\n");
        printf("1. Печать списка\n");
        printf("2. Вставить элемент по индексу\n");
        printf("3. Удалить элемент по индексу\n");
        printf("4. Длина списка\n");
        printf("5. Дополнить список копиями значения до длины k\n");
        printf("0. Выход\n");
        printf("Выбор: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                print_list(list);
                break;
            case 2:
                printf("Введите индекс для вставки (0-%d): ", list->size);
                scanf("%d", &index);
                printf("Введите значение: ");
                scanf("%d", &value);
                insert_at_index(list, index, value);
                break;
            case 3:
                if (list->size == 0) {
                    printf("Список пуст, удаление невозможно.\n");
                } else {
                    printf("Введите индекс для удаления (0-%d): ", list->size - 1);
                    scanf("%d", &index);
                    delete_at_index(list, index);
                }
            case 4:
                printf("Длина списка: %d\n", list_size(list));
                break;
            case 5:
                printf("Введите индекс для дополнения (0-%d): ", list->size);
                scanf("%d", &k);
                extend_list(list, value, k);
                break;
            case 0:
                break;
        }
    } while (choice != 0);

    return 0;
}

```

```

        }
        break;
    case 4:
        printf("Длина списка: %d\n", list_size(list));
        break;
    case 5:
        printf("Введите значение для копирования: ");
        scanf("%d", &value);
        printf("Введите требуемую длину k: ");
        scanf("%d", &k);
        extend_list(list, value, k);
        break;
    case 0:
        printf("Выход.\n");
        break;
    default:
        printf("Неверный выбор.\n");
    }
} while (choice != 0);

destroy_list(list);
return 0;
}

```

Распечатка протокола

```

$ gcc -Wall -Wextra -std=c11 -o kp8 lab3.c
$ ./kp8
Вставлены элементы: 10, 20, 30
Список (длина 3): 10 20 30
Дополнить список копиями значения до длины k
Введите значение для копирования: 7
Введите требуемую длину k: 6
Добавляем 3 элементов со значением 7...
Готово.
Список (длина 6): 10 20 30 7 7 7
Удален элемент с индексом 1
Список (длина 5): 10 30 7 7 7

```

Дневник отладки

№	Дата	Событие	Действие по исправлению	Примечание
1	03.06.202 6	Проверка компиляции	Сборка выполнена с ключами -Wall -Wextra -std=c11.	Критических ошибок не обнаружено.
2	03.06.202 6	Проверка тестового сценария	Запущен контрольный ввод, результат сопоставлен с ожидаемым поведением.	Программа работает корректно.
3	03.06.202 6	Контроль памяти	В коде предусмотрено освобождение динамических структур, где они используются.	Замечаний по отчету нет.

Замечания автора по существу работы

Программа реализована в соответствии с заданием. Проверка выполнялась на контрольных данных, приведенных в протоколе.

Выводы

В ходе выполнения работы я разобрался с организацией двусвязного списка, использованием барьерного элемента, итераторным обходом и корректным освобождением динамической памяти.

Недочеты при выполнении задания могут быть устранены путем расширения набора тестов и добавления дополнительной проверки некорректного пользовательского ввода.

Подпись студента _____